

User-defined Functions in C

A **function** in C is a block of organized, reusable code that is used to perform a single related action. In any C program, there are one or more functions – classified as library functions and user-defined functions.

There are two types of functions in C –

- Library functions
- User-defined functions

Any C compiler (e.g. GCC compiler, Clang, MSVC compiler, etc.) is distributed with a number precompiled header files (`stdio.h`, `math.h`, etc.), each consisting of one or more predefined library functions such as `printf()`, `scanf()`, `pow()`, `sqrt()`, etc. To be able to use the library function, the corresponding header file must be made available with the **#include** directive.

However, if you don't find a suitable library function to serve your purpose, then you can define a customized function for the program. Normally, we find a C program with a **main() function**. Obviously, the `main()` function is a user-defined function, as it contains the instructions provided by the user. It can of course call the other library or user-defined functions.

What is User-Defined Function in C?

User-defined function is defined by the user to perform specific task to achieve the code reusability and modularity. To create and use the user-defined function, you do not need use any built-in library. These functions can be created either in the same program or in user-defined header file.

Creating a User-defined Function

For creating a user-defined function, first you need to understand the purpose of the function, that is, what do you want the function to do?

To create a user-defined function, you need to know about the following three parts of a function:

- Function declaration
- Function definition
- Function calling

Declaration of User-defined Function

In C language, it is necessary to provide the declaration of the prototype of any function. The prototype of a library function is present in the corresponding header file.

For a user-defined function, its prototype is present in the current program. The definition of a function and its prototype declaration should match.

Syntax

If you wish to define a function called `add()` that performs the addition of two integer arguments and returns the value as an integer, then the function declaration would be as follows –

```
int add(int, int);
```

Definition of User-defined Function

The definition of a function and its prototype declaration should match. The definition consists of a function header that matches the declaration and a function body.

Syntax

```
return_type function_name(arg1, arg2, ...){

    // Function body;

    return val;

}
```

Example

Using this template, you can write the user-defined function `add()` as follows –

```
int add(int a, int b){
    int c;
    c = a + b;
    return c;
}
```

Note that the order of definition of user-defined functions is not important in a C program. However, its prototype must be declared before calling the function.

In a program, the `main()` function is always the entry point, irrespective of whether it is the first function or not. We needn't provide the prototype declaration of the `main()` function.

Calling a User-defined Function

To call a function, you should use a statement that complies with the declaration of the function prototype. If the function is defined to receive a certain number of arguments, then the same number and type of arguments must be passed to call that function.

Example

The following statement calls the `add()` function that we defined above –

```
int result = add(10, 20);
```

Explore our **latest online courses** and learn new skills at your own pace. Enroll and become a certified expert to boost your career.

Example of User-Defined Function

In this example, we are create two user-defined functions **`add()`** and **`sub()`** to find the addition and subtraction of the given numbers.


[Open Compiler](#)

```
// C program to demonstrate an example of
// user-defined function
#include <stdio.h>

// Function declarations
int add(int, int);
int sub(int, int);

// Function definitions
int add(int a, int b) {
    return (a + b);
}

int sub(int a, int b) {
```

```

    return (a - b);
}

int main() {
    // Declaring two integer variables to
    // store the numbers
    // and resultant variables to store the result
    int num1 = 36, num2 = 24;
    int res_add, res_sub;

    // Calling the functions
    res_add = add(num1, num2);
    res_sub = sub(num1, num2);

    // Printing the results
    printf("Addition is : %d\n", res_add);
    printf("Subtraction is : %d\n", res_sub);

    return 0;
}

```

```

Addition is : 60
Subtraction is : 12

```

Formal and Actual Arguments in User-Defined Function

When a function is defined with arguments, the arguments in the parenthesis in front of the function name are called **formal arguments**. In the above example, the function is defined with "int a" and "int b" arguments; they are formal arguments.

When the function is called, the arguments passed to it are called the **actual arguments**. In the example below, the variables "x" and "y" are the actual arguments.

```

int x = 10, y = 20;
int result = add(x, y);

```

A user-defined function may be defined to have any type of variables as formal arguments. It includes primary types (int, float, char), array, pointer, or struct/union type variables.

A function should return a value to the calling environment. By default, the return type of a function is **int** type. However, it can return any data type – primary type, array, pointer, or a struct as well as a pointer. You can even define a function that returns a **void** type.

Example

If either the number or the type of actual and formal arguments or the return type as in the forward declaration of a function and its definition don't match, then the compiler reports an error.

Look at the example below –


[Open Compiler](#)

```
#include <stdio.h>

float divide (int, int);

int main(){

    int x = 15, y = 5;

    float z = divide (x, y);

    printf("%f", z);

    return 0;
}

int divide (int a, int b){

    int c = a/b;

    return c;
}
```

Output

In the code above, the declaration of the divide() function doesn't match with its definition, hence the compiler shows the following error –

error: conflicting types for 'divide'

In C, any function can call any other function, any number of times. A function can call itself too. Such a self-calling function is called a **recursive function**.

The following program calls the main() function from inside main() itself –

```
#include <stdio.h>

int main(){

    printf("Hello");

    main();

    return 0;
}
```

When executed, the program goes into an infinite loop. In practice, **recursion** has to be used so that the program eventually terminates.